



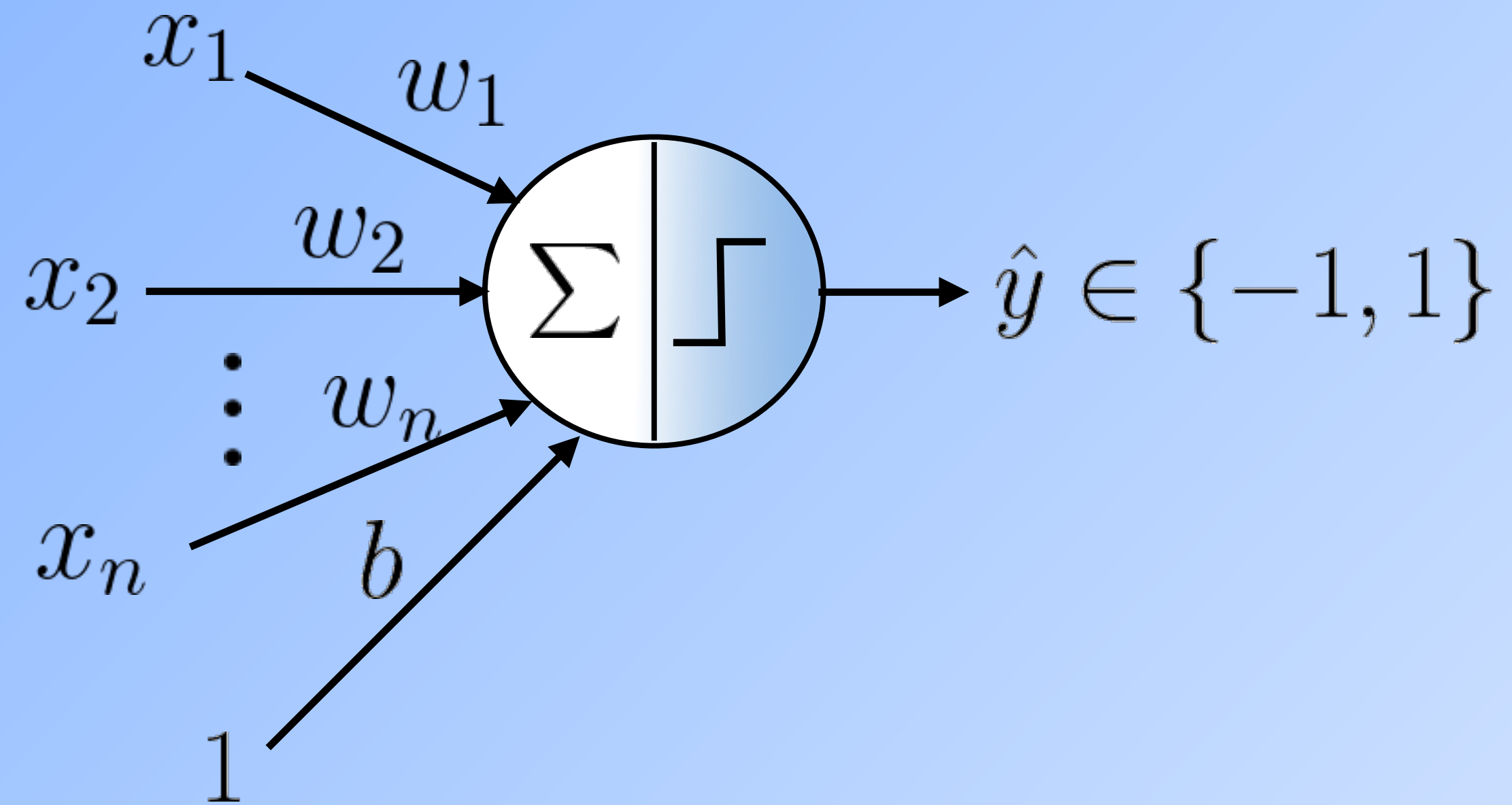
AI61201: Deep Learning Foundations and Applications

Module 1: Multi-layer Perceptrons

Dr. Somdyuti Paul

The Perceptron

- The perceptron is a linear, binary classifier that was used as building blocks of early neural networks.



Input: $\mathbf{x} = [x_1, x_2, \dots, x_n, 1]^T$

Weights: $\mathbf{w} = [w_1, w_2, \dots, w_n, b]^T$

Prediction: $\hat{y} = \text{sign}(\mathbf{w}^T \mathbf{x})$



The Mark I perceptron machine from 1958 designed using hardware circuits

The Perceptron Algorithm

Let a ground truth label be y , and the corresponding prediction $\hat{y}$.

Parameter update rule:

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t + \alpha y \mathbf{x} \text{ if } y\hat{y} < 0 \\ &= \mathbf{w}_t \text{ otherwise} \end{aligned}$$

Intuition:

Case 1: $y = +1, \hat{y} = -1$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \mathbf{x}$$

$$\mathbf{w}_{t+1}^T \mathbf{x} = \mathbf{w}_t^T \mathbf{x} + \alpha \mathbf{x}^T \mathbf{x} > \mathbf{w}_t^T \mathbf{x}$$

Case 2: $y = -1, \hat{y} = +1$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{x}$$

$$\mathbf{w}_{t+1}^T \mathbf{x} = \mathbf{w}_t^T \mathbf{x} - \alpha \mathbf{x}^T \mathbf{x} < \mathbf{w}_t^T \mathbf{x}$$

The Perceptron Algorithm

Training data: $(\mathbf{x}^{(i)}, y^{(i)}) \forall i \in \{1, 2, \dots, M\}$

Learning rate: α

Result: $\mathbf{w}^*$

Initialize:

$$\mathbf{w}_0 \leftarrow \mathbf{0};$$

$$t = 0;$$

repeat:

for $i = 1, 2, \dots, M$, **do**

$$\hat{y}^{(i)} = \text{sign}(\mathbf{w}_t^T \mathbf{x}^{(i)});$$

if $y^{(i)}\hat{y}^{(i)} < 0$ **then**

$$\mathbf{w}_{t+1} \leftarrow \mathbf{w}_t + \alpha y^{(i)} \mathbf{x}^{(i)};$$

$$t \leftarrow t + 1;$$

end

end

until convergence

15 lines

Convergence of the Perceptron Algorithm

- The perceptron algorithm converges in a finite number of steps when the training data is linearly separable.

Proof:

If the training data is linearly separable $\exists \mathbf{w}^* \in \mathbb{R}^n$ that perfectly classifies the data. Also. Let $\gamma = \min_i y^{(i)}(\mathbf{w}^{*T} \mathbf{x}^{(i)})$ and $\|\mathbf{x}^i\| \leq R$

After the k^{th} weight update, we have,

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha y^{(i)} \mathbf{x}^{(i)}$$

$$\begin{aligned} \mathbf{w}_t^T \mathbf{w}^* &= (\mathbf{w}_{t-1} + \alpha y^{(i)} \mathbf{x}^{(i)})^T \mathbf{w}^* \\ &= \mathbf{w}_{t-1}^T \mathbf{w}^* + \alpha y^{(i)} \mathbf{x}^{(i)T} \mathbf{w}^* \\ &\geq \mathbf{w}_{t-1}^T \mathbf{w}^* + \alpha \gamma \end{aligned}$$

As $\mathbf{w}_0 = \mathbf{0}$, by induction, we have $\mathbf{w}_k^T \mathbf{w}^* \geq k\alpha\gamma$.

Here we have 3 things pre-defined:

1> there exists $\mathbf{w}^*$ which is n-dimensional hyperplane

2> $\gamma = \min_i (y_i * \mathbf{w}^{*T} \mathbf{x}_i)$

3> $\|\mathbf{x}_i\| \leq R$, all the feature vectors of all Training examples are bounded

Convergence of the Perceptron Algorithm

Proof:

Using Cauchy-Schwarz inequality,

$$\begin{aligned} (\mathbf{w}_k^T \mathbf{w}^*)^2 &\leq \|\mathbf{w}_k\|^2 \|\mathbf{w}^*\|^2 \\ \therefore k^2 \alpha^2 \gamma^2 &\leq (\mathbf{w}_k^T \mathbf{w}^*)^2 \leq \|\mathbf{w}_k\|^2 \|\mathbf{w}^*\|^2 \end{aligned}$$

Also,

$$\begin{aligned} \|\mathbf{w}_k\|^2 &= (\mathbf{w}_{k-1} + \alpha y^{(i)} \mathbf{x}^{(i)})^T (\mathbf{w}_{k-1} + \alpha y^{(i)} \mathbf{x}^{(i)}) \\ &= \|\mathbf{w}_{k-1}\|^2 + 2\alpha y^{(i)} \mathbf{w}_{k-1}^T \mathbf{x}^{(i)} + \alpha^2 \|\mathbf{x}^{(i)}\|^2 \\ &\leq \|\mathbf{w}_{k-1}\|^2 + \alpha^2 R^2 \end{aligned}$$

Since $\mathbf{w}_0 = \mathbf{0}$, by induction, we have $\|\mathbf{w}_k\|^2 \leq k\alpha^2 R^2$

So, we have:

$$k^2 \alpha^2 \gamma^2 \leq k\alpha^2 R^2 \|\mathbf{w}^*\|^2$$

$$\text{i.e. } k \leq \left(\frac{R}{\gamma}\right)^2 \|\mathbf{w}^*\|^2.$$

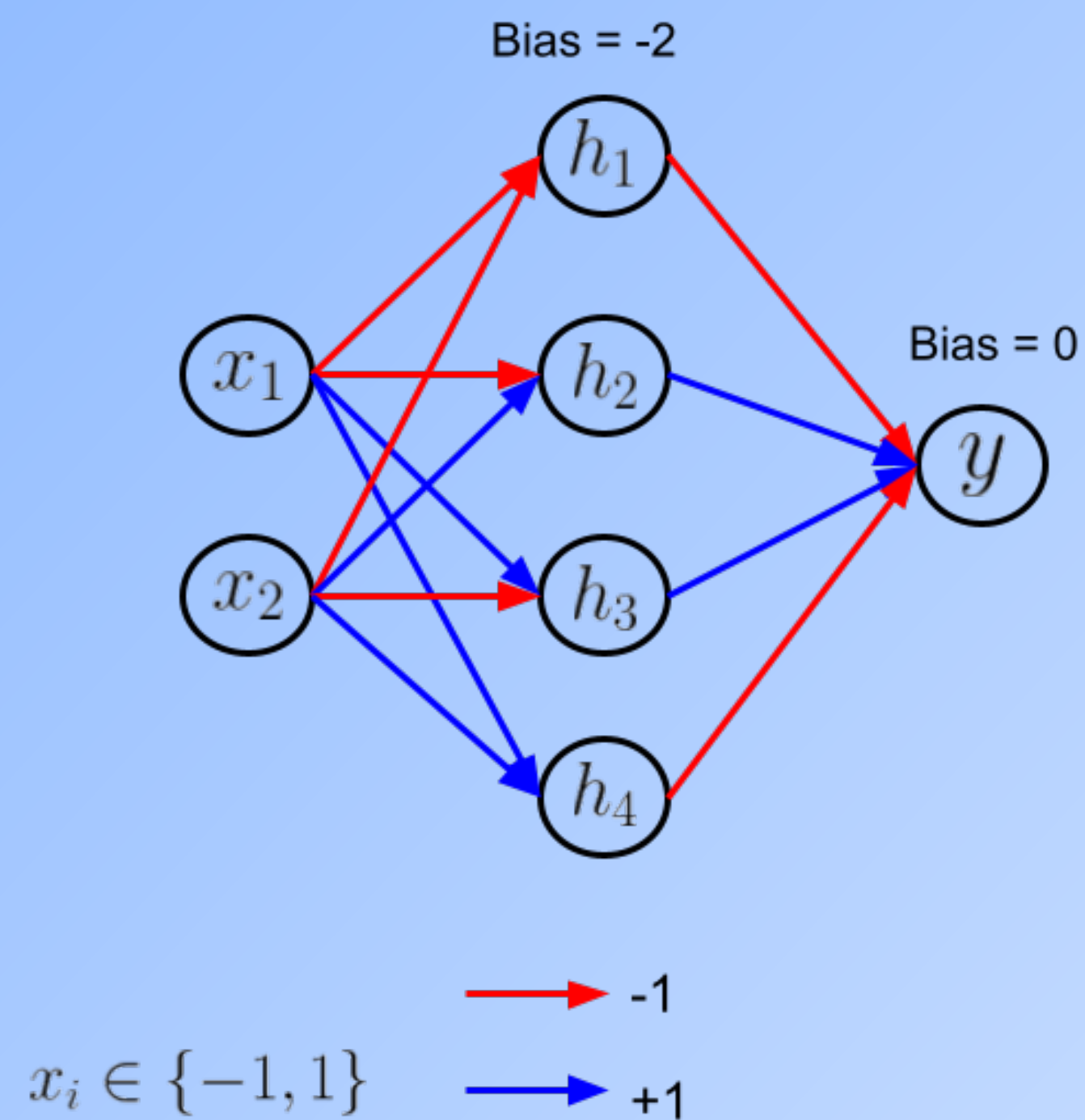
Limitations of Perceptrons

- Cannot classify non-linearly separable data - unable to implement simple boolean functions like XOR
- Outputs are binary valued due to the activation function being a step function.
- Limited to a single layer
- Sensitive to outliers

To overcome aforementioned limitations:

- Use multi-layer neural perceptrons (MLPs)
- Use other activation functions.

Implementing the XOR Function with Multi-layer Perceptron



x_1	x_2	h_1	h_2	h_3	h_4	y
-1	-1	1	-1	-1	-1	-1
-1	1	-1	1	-1	-1	1
1	-1	-1	-1	1	-1	1
1	1	-1	-1	-1	1	-1

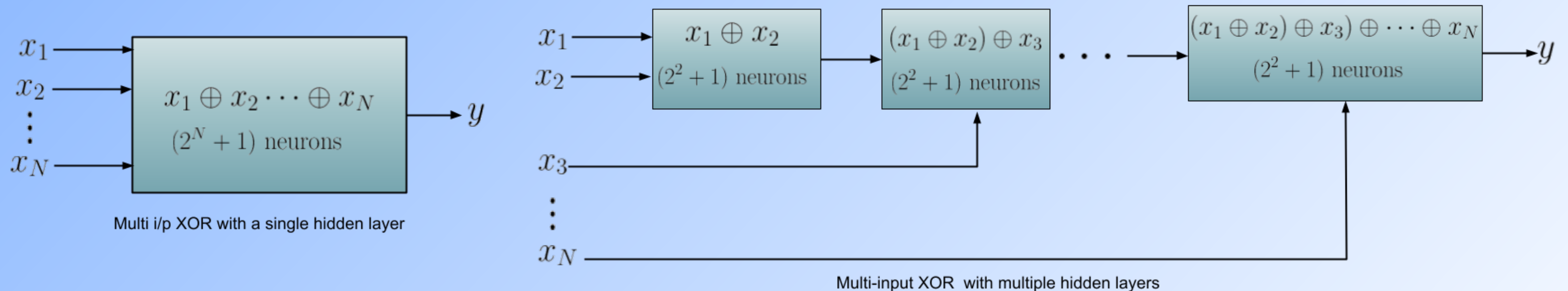
Thus by adding a hidden layer, we were able to implement the XOR function!

Representation Power of Multi-layer Perceptrons

- Any arbitrary boolean function having N inputs can be represented by an MLP having 2^N neurons in the hidden layer i.e. a MLP with a single hidden layer acts as an universal boolean machine.
- Intuition
 - N boolean inputs can have 2^N logical combinations.
 - Each unit in the hidden layer can be designed to activate for a single input combination, followed by the output layer which combines these activations using weighted sum.
- The disadvantage is that the number of neurons in the hidden layer increases exponentially with the number of inputs.

Need for Deeper Neural Network Architectures

- Consider a XOR gate having $N > 2$ inputs
- An MLP with a single hidden layer would require $2^N + 1$ neurons, i.e. the number of neurons increases exponentially with input size.
- What happens if we increase the number of layers?



- Total # of units for the network with a single hidden layer = $2^N + 1$ (increases exponentially with increase in N)
- Total # of units for the network with multiple hidden layers = $(2^2 + 1) \cdot (N - 1)$ (increases linearly with increase in N)

Need for Deeper Network Architectures

Advantages of deep networks

- Repeated compositions of non-linear functions allow the networks to exploit regularities in data and learn multiple levels of hierarchical abstractions.
- Adding more layers acts as a form of regularization and thus improves generalization.
- The non-linear mappings enabled by activation functions serve to make the data linearly separable.

Disadvantage of Deep Networks

- Deeper networks are harder to train (due to vanishing and exploding gradients).
- It is harder to interpret the features learned by deeper layers.